

DataBridge

Architektur & Skript-Aufrufkette

Hochdetaillierte technische Dokumentation des aktuellen Stands

Projekt	Master-FuE-Projekt 26ss_CB_DataBridge (HS Karlsruhe / Univ. Oviedo)
Team	Marcel Thiel (Wissensnetz), Kollege A (Wrapper), Kollege B (Mediator)
Betreuung	Prof. Catherina Burghart
Repository	F-E_Projekt1 · Branch main
Stand	26. August 2026
Zweck dieses Dokuments	Nachvollziehbarkeit der kompletten Skript-Kette; Autor und Reifegrad jedes Skripts

Inhalt

1 Zweck & Geltungsbereich

Dieses Dokument beschreibt, wie die DataBridge-Komponenten zur Laufzeit tatsächlich zusammenspielen — von dem einen Startbefehl bis zur laufenden Oberfläche. Für jedes beteiligte Skript ist festgehalten, **von welchem Kollegen** es stammt und **welchen Reifegrad** es hat (voll ausgebaut, bewusst vereinfachter Prototyp oder reiner Platzhalter). Grundlage ist ausschließlich der aktuelle Quellcode im Repository, nicht die Planungsdokumente.

Wichtige Architektur-Randbedingung (bleibt über das ganze Dokument gültig): Die drei Komponenten sind **strikt getrennt**. Das Wissensnetz spricht ausschließlich per HTTP/SPARQL mit dem Triple-Store und importiert keinen Code aus `mediator/` oder `wrappers/`. Die Abhängigkeitsrichtung ist ausschließlich Mediator → Wissensnetz.

2 Die drei Komponenten und ihre Autoren

Der Code gliedert sich in drei Verantwortungsbereiche. Die Ordnerzuordnung ist eindeutig:

Bereich / Autor	Ordner	Rolle in der Kette
Kollege A • Wrapper	<code>wrappers/gdc/</code>	Kapselt den Zugriff auf die GDC/TCGA-REST-API. Liefert rohe Metadaten (JSON).
Kollege B • Mediator	<code>mediator/</code>	FastAPI-Service. Nimmt Anfragen entgegen, ruft den Wrapper und übersetzt GDC-JSON → RDF/Turtle.
Marcel • Wissensnetz	<code>wissensnetz/</code> , <code>scripts/</code> , <code>start_all.ps1</code>	Triple-Store-Anbindung, SPARQL-Anreicherung, Rückkanal, MP-lite-Prototyp und die gesamte Orchestrierung.

Hinweis zur Orchestrierung: Die Skripte `start_all.ps1`, `stop_all.ps1`, `scripts/load_gdc.py` und `scripts/graph_view.py` sind Projekt-Glue und gehören zu Marcells Bereich. Sie rufen den Mediator (B) nur über HTTP auf und halten die Komponentengrenze ein.

3 Reifegrad-Legende

Jede Datei ist im weiteren Verlauf mit einem der drei Etiketten markiert:

[PRODUKTIV] – im Prototyp-Rahmen voll lauffähig und gegen echte Systeme (GDC-API, Fuseki) verifiziert.

[PROTOTYP / TESTFUNKTION] – funktioniert, ist aber bewusst vereinfacht bzw. eine Demonstration (z. B. synthetische Daten statt echtem Tool).

[PLATZHALTER - NICHT AUSGEBAUT] – Grundgerüst / Signatur vorhanden, aber noch ohne echte Umsetzung (z. B. `NotImplementedError`, ungenutztes Cache-Tier, leerer Service).

4 Die Laufkette Schritt für Schritt

Der einzige nötige Startbefehl ist `./start_all.ps1` (aus dem Projekt-Root, mit aktivierter Conda-Env F+E). Dieses Skript ist der rote Faden der gesamten Kette. Es ruft der Reihe nach die folgenden Schritte auf. Jeder Schritt nennt das ausführende Skript, den Autor und den Reifegrad.

4.1 Abhängigkeiten sicherstellen

Autor: **Marcel • Wissensnetz** Status: **[PRODUKTIV]**

- Prüft, ob der Befehl `wissensnetz` existiert und ob `bokeh`, `pyvis`, `requests`, `rdflib` importierbar sind.
- Falls nicht: `pip install -r requirements.txt` (installiert u. a. die lokalen Pakete `wissensnetz` und `wrappers editable`).

4.2 Docker-Engine prüfen / starten

Autor: **Marcel • Wissensnetz** Status: **[PRODUKTIV]**

- Führt `docker info` aus; läuft Docker nicht, wird Docker Desktop gestartet und bis zu 60×3 s auf Bereitschaft gewartet.

4.3 Triple-Store (Fuseki) starten

Autor: **Marcel • Wissensnetz** Status: **[PRODUKTIV]**

- Startet den Container `graph-db` (Image `stain/jena-fuseki`) via `docker compose up -d graph-db`.
- Wartet auf `http://localhost:3030/$/ping`. Das Dataset `databridge` legt das Image über `FUSEKI_DATASET_1` persistent im Volume `graph-db-data` an.

```
docker compose up -d graph-db # Fuseki, Port 3030, Login admin/admin
```

4.4 Wissensnetz initialisieren

Autor: **Marcel • Wissensnetz** Status: **[PRODUKTIV]**

- Ruft `wissensnetz init` → `wissensnetz/src/wissensnetz/init.py :: initialize()`.
- Stellt das Dataset sicher (Fuseki-Admin-API, idempotent) und lädt die TBox `ontology/databridge-core.ttl` sowie das Rückkanal-Vokabular `ontology/feedback.ttl` per Graph Store Protocol — beides idempotent (überspringt, wenn bereits vorhanden).

4.5 Mediator (FastAPI) starten

Autor: **Kollege B • Mediator** Status: **[PRODUKTIV]**

- Startet in einem eigenen Fenster: `python -m uvicorn app.main:app --port 8000` im Ordner `mediator/`.
- Wartet auf `http://localhost:8000/health`. **Hinweis:** `start_all` nutzt bewusst den lokalen `uvicorn`-Prozess, nicht den `Compose-Service mediator` (der existiert zusätzlich in `docker-compose.yml`, wird hier aber nicht hochgefahren).

4.6 TCGA/GDC-Daten abrufen und laden

Autor: **Marcel • Wissensnetz** Status: **[PRODUKTIV]**

- Ruft `python scripts/load_gdc.py --project TCGA-BRCA --size 50`. Dieses Skript orchestriert die eigentliche Datenaufnahme (Details in Kapitel 5).
- Kurz: es fragt beim Mediator `POST /transform` an (GDC → Turtle) und lädt das zurückgegebene Turtle anschließend selbst per `GraphStore.load_turtle()` in Fuseki.

4.7 Wissensnetz visualisieren (pyvis)

Autor: **Marcel • Wissensnetz** Status: **[PRODUKTIV]**

- Ruft `python scripts/graph_view.py --limit 500`.
- Zieht per SPARQL Tripel aus `Default-` und `Named-Graphs`, baut ein interaktives HTML-Netz (`graph_view.html`), farbcodiert nach Bereich (`Schema/Instanz/Rückkanal/extern`) und öffnet es im Browser.

- Degradiert graziös: fehlt pyvis, wird der Schritt übersprungen statt abzurechnen.

4.8 Oberfläche MP-lite starten

Autor: **Marcel · Wissensnetz** Status: **[PROTOTYP / TESTFUNKTION]**

- Startet bokeh serve --show --port 5006 wissensnetz/prototype/mp_lite/app.py — der Browser öffnet den Prototyp.
- Beim Laden: initialize(store) + Beispiel-ABox cases_brca_sample.ttl. Punkt-Selektion → Kontext (②); Hypothese speichern → Rückkanal (③).
- Bewusst vereinfacht (siehe Kapitel 6.4 / 9): synthetische Punkte und zwei Zufalls-Layouts statt echter MP-Daten.

4.9 Herunterfahren (Strg+C)

Autor: **Marcel · Wissensnetz** Status: **[PRODUKTIV]**

- Bokeh läuft im Vordergrund; Strg+C löst den finally-Block aus: Stop-All beendet Ports 8000 und 5006, fährt docker compose down und schließt danach das Fenster (Stop-Process \$PID).
- Manuell / nach -NoUi: ./stop_all.ps1 macht dasselbe ohne das Fenster zu schließen.

4.10 Kette auf einen Blick

```
start_all.ps1 (Marcel · Orchestrierung)
|
|-- 4.1 pip install -r requirements.txt      (falls noetig)
|-- 4.2 docker info / Docker Desktop
|-- 4.3 docker compose up -d graph-db -----> [Fuseki : 3030] (Container)
|-- 4.4 wissensnetz init -----> laedt TBox + feedback.ttl
|-- 4.5 uvicorn app.main:app --port 8000 ----> [Mediator : 8000] (Kollege B)
|-- 4.6 python scripts/load_gdc.py
|   |-- POST http://localhost:8000/transform (-> Mediator B)
|   |   |-- GDCWrapper.search('cases', ...) (-> Wrapper A -> GDC-API)
|   |   |-- cases_to_graph + serialize_with_provenance (Turtle)
|   |-- GraphStore.load_turtle(turtle)      (-> Fuseki GSP)
|-- 4.7 python scripts/graph_view.py -----> SPARQL -> graph_view.html
|-- 4.8 bokeh serve mp_lite/app.py -----> [MP-lite : 5006] (Prototyp)
|-- 4.9 Strg+C -> Stop-All (kill 8000/5006, docker compose down)
```

5 Detaillierter Datenfluss: der Kern-Endpunkt /transform

Der Endpunkt POST /transform (Datei mediator/app/main.py, Autor B) ist das Herz der Datenaufnahme. Er kann auf **zwei Wegen** enden — die aktuelle Kette nutzt Weg A.

5.1 Ablauf innerhalb von /transform

- Wenn keine rohen cases mitgegeben werden: `GDCWrapper.search('cases', fields=TRANSFORM_CASE_FIELDS, size=...)` holt sie live von der GDC-API (Autor A).
- `load_alignment_table(alignment_path('ncit_primary_diagnosis.json'))` lädt die Enum→NCIt-Tabelle (Schlüssel mit führendem _ werden ignoriert).
- `cases_to_graph(cases, alignment=...)` erzeugt die ABox (db:Case/Project/Demographic/Diagnosis). Bei Alignment-Treffer wird zusätzlich db:primaryDiagnosis (NCIt-IRI) gesetzt und eine RDF-star-Provenienz vorgemerkt.
- `serialize_with_provenance(...)` serialisiert nach Turtle und hängt die RDF-star-Blöcke (Provenienz/Konfidenz) als Text an.

5.2 Weg A — zweistufig (das nutzt start_all)

- `scripts/load_gdc.py` ruft /transform ohne load, bekommt das Turtle zurück und lädt es selbst mit `GraphStore.load_turtle()` in Fuseki. Vorteil: kein Fuseki-Zugriff nötig, wenn der Mediator lokal läuft.

5.3 Weg B — direkt (Alternative, hier nicht aktiv)

- Wird /transform mit "load": true aufgerufen, schreibt der Mediator das Turtle selbst per Graph Store Protocol in graph-db. Im Compose-Netz zeigt dafür `GRAPH_DB_URL=http://graph-db:3030`. Diese Variante ist implementiert, wird von der aktuellen Startkette aber nicht verwendet.

6 Komponenten-Referenz (Datei für Datei)

Gruppieren nach Autor. Jede Datei mit Pfad, Reifegrad und Kernfunktion.

6.1 Kollege A — Wrapper (wrappers/gdc/)

wrappers/gdc/client.py

Kollege A • Wrapper [PRODUKTIV]

- GDCWrapper.query / search / get_schema / build_manifest — Metadaten- und Bulk-Tier gegen die echte GDC-API; live verifiziert (TCGA-BRCA / RNA-Seq / open).
- Aktiv in der Kette: search('cases', ...) wird von /transform genutzt.

wrappers/gdc/client.py :: download_via_gdc_client()

Kollege A • Wrapper [PLATZHALTER - NICHT AUSGEBAUT]

- Übergabe eines Manifests an das externe Tool gdc-client. Ist es nicht installiert, wird status="not_run" gemeldet — reiner Platzhalter für den späteren Bulk-Download.

wrappers/gdc/client.py :: to_anndata()

Kollege A • Wrapper [PLATZHALTER - NICHT AUSGEBAUT]

- Bewusst NotImplementedError — die Umwandlung nach anndata/.h5ad ist ausdrücklich kein Teil des Wrappers.

wrappers/gdc/cache.py

Kollege A • Wrapper [PROTOTYP / TESTFUNKTION]

- Tier 1 (Recipes) wird bei jeder query() beschrieben. Tier 2 (materialized/anndata) und Tier 3 (raw) sind Datei-Grundgerüst, aber noch ohne echte Nutzung.

6.2 Kollege B — Mediator (mediator/)

mediator/app/main.py

Kollege B • Mediator [PRODUKTIV]

- /health, /query, /schema/{endpoint}, /manifest, /transform, /ontology — vollständig funktionsfähig.
- Hält je eine geteilte GDCWrapper- und GraphStore-Instanz (lazy).

mediator/app/semantic/mapping.py

Kollege B • Mediator [PRODUKTIV]

- cases_to_graph() + serialize_with_provenance() — Kern-Ausschnitt case/project/demographic/diagnosis nach RDF/OWL, RDF-star für Alignment-Provenienz. Global-as-View (nur GDC).

mediator/app/schemas.py

Kollege B • Mediator [PRODUKTIV]

- Pydantic-Requests (QueryRequest, ManifestRequest, TransformRequest) — vereinfachte Parameter statt vollem GDC-filters-Schema.

mediator/app/semantic/paths.py

Kollege B • Mediator [PRODUKTIV]

- Pfad-Auflösung für Ontologie/Alignment über DATABRIDGE_ONTOLOGY_DIR mit lokalem Fallback.

mediator/app/main.py :: anndata/.h5ad-Konvertierung

Kollege B • Mediator [PLATZHALTER - NICHT AUSGEBAUT]

- Im Modul-Docstring als „folgt in späteren Schritten“ angekündigt; die numerische Messmatrix-Schicht existiert noch nicht.

6.3 Marcel — Wissensnetz-Paket (wissensnetz/src/wissensnetz/)

config.py

Marcel • Wissensnetz [PRODUKTIV]

- Fuseki-Settings + Namespaces + wiederverwendbarer PREFIXES-Block; alles aus ENV.

graphstore.py

Marcel · Wissensnetz [PRODUKTIV]

- load_turtle / query / ask / update + Dataset-Verwaltung. Turtle wird **roh** gesendet (kein rdflib-Roundtrip), damit RDF-star erhalten bleibt.

init.py

Marcel · Wissensnetz [PRODUKTIV]

- initialize() — Dataset sicherstellen + TBox + Rückkanal-Vokabular idempotent laden.

enrichment.py (Aufgabe 3, ② Lesen)

Marcel · Wissensnetz [PRODUKTIV]

- subclasses/superclasses (rdfs:subClassOf*), case_context/diagnosis_context — reine Lesefunktionen.
- Achtung: Der Modul-Docstring sagt noch „Alignment-Tabelle ist derzeit leer“. Das ist **veraltet** — die Tabelle enthält inzwischen verifizierte NCIt-Einträge (C4194, C7950, C7688).

feedback.py (Aufgabe 4, ③ Schreiben)

Marcel · Wissensnetz [PRODUKTIV]

- SelectionEvent/Hypothesis, selection_to_sparql (SPARQL-star INSERT), write_feedback/list_findings/reclassifications — Named Graph pro Nutzer.

cli.py

Marcel · Wissensnetz [PRODUKTIV]

- CLI wissensnetz status|init|load|query|hierarchy|context|feedback|findings.

ontology/alignment/ncit_primary_diagnosis.json

Marcel · Wissensnetz [PROTOTYP / TESTFUNKTION]

- Enum→NCIt-Tabelle. Aktuell 3 verifizierte Einträge (duktal C4194, lobulär C7950, gemischt C7688) — bewusst kleine Testbasis, wird inkrementell erweitert.

6.4 Marcel — Orchestrierung & Prototyp

start_all.ps1 / stop_all.ps1

Marcel · Wissensnetz [PRODUKTIV]

- Kompletter Start/Stop der Kette (Kapitel 4). Reines ASCII wegen Windows-PowerShell-5.1-Encoding.

scripts/load_gdc.py

Marcel · Wissensnetz [PRODUKTIV]

- Zweistufige Datenaufnahme (Weg A, Kapitel 5.2). Prüft Mediator und Fuseki vorab.

scripts/graph_view.py

Marcel · Wissensnetz [PRODUKTIV]

- pyvis-Visualisierung aus Fuseki; nutzt nur GraphStore (SPARQL), keine Kopplung an Mediator/Wrapper.

wissensnetz/prototype/mp_lite/app.py

Marcel · Wissensnetz [PROTOTYP / TESTFUNKTION]

- Bokeh-Prototyp, der den Loop MP ↔ Wissensnetz vorführt (② Kontext, ③ Rückkanal).
- Bewusst vereinfacht: 4 echte Barcodes aus der Fixture + 20 synthetische Punkte (SYN-0001...0020), zwei deterministische Zufalls-Layouts als „Gene ↔ miRNA“-Morph — kein echtes MP-Backend. Zielweg später: die echte MP-Web-Version.

7 Reifegrad-Gesamtmatrix

Datei / Modul	Autor	Status	Offen / Anmerkung
wrappers/gdc/client.py (query/search/schema/manifest)	Kollege A • Wrapper	PRODUKTIV	—
wrappers/gdc/client.py :: download_via_gdc_client	Kollege A • Wrapper	PLATZHALTER	Bulk-Download via gdc-client noch nicht aktiv
wrappers/gdc/client.py :: to_anndata	Kollege A • Wrapper	PLATZHALTER	Absichtlich NotImplemented (gehört zum Mediator)
wrappers/gdc/cache.py (Recipes)	Kollege A • Wrapper	PRODUKTIV	—
wrappers/gdc/cache.py (Materialized/Raw)	Kollege A • Wrapper	PROTOTYP	Tier 2/3 nur Grundgerüst
mediator/app/main.py (Endpunkte)	Kollege B • Mediator	PRODUKTIV	—
mediator/app/semantic/mapping.py	Kollege B • Mediator	PRODUKTIV	Nur Kern-Ausschnitt; weitere GDC-Nodes offen
mediator: anndata/.h5ad-Konvertierung	Kollege B • Mediator	PLATZHALTER	Numerische Messmatrix-Schicht fehlt noch
wissensnetz: config/graphstore/init/enrichment/feedback/cli	Marcel • Wissensnetz	PRODUKTIV	Aufgabe 1–4 umgesetzt
wissensnetz/ontology/alignment/ ncit_primary_diagnosis.json	Marcel • Wissensnetz	PROTOTYP	3 verifizierte Einträge (Testbasis)
scripts/load_gdc.py, scripts/graph_view.py	Marcel • Wissensnetz	PRODUKTIV	—
start_all.ps1 / stop_all.ps1	Marcel • Wissensnetz	PRODUKTIV	—
wissensnetz/prototype/mp_lite/ app.py	Marcel • Wissensnetz	PROTOTYP	Synthetische Daten; echtes MP-Backend später
frontend/	—	PLATZHALTER	Leerer Platzhalter, kein Compose-Service

8 Test- & Smoke-Skripte (nicht Teil der Laufkette)

Diese Skripte werden nicht von start_all.ps1 aufgerufen. Sie dienen der manuellen/CI-Prüfung und dem Beispiel:

wrappers/gdc/scripts/check_connection.py

Kollege A • Wrapper [PROTOTYP / TESTFUNKTION]

- Smoke-Test: prüft echten GDC-API-Zugriff (search + get_schema). Exit 0/1, kein Mocking.

mediator/scripts/check_mediator.py

Kollege B • Mediator [PROTOTYP / TESTFUNKTION]

- End-to-End-Smoke-Test in-process (fastapi.testclient): /health, /ontology, /query, /transform gegen die reale GDC-API.

mediator/scripts/example_gdc_to_rdf.py

Kollege B • Mediator [PROTOTYP / TESTFUNKTION]

- Beispiel ohne laufenden Service: Beispieldaten cases_brca_sample.json → Turtle-Datei; nutzt dieselbe Mapping-Logik wie /transform.

wissensnetz/tests/ (pytest)

Marcel • Wissensnetz [PROTOTYP / TESTFUNKTION]

- Unit-Tests für graphstore/init/enrichment/feedback. Brauchen ein laufendes Fuseki, sonst werden sie übersprungen.

9 Bewusste Vereinfachungen & Platzhalter (Zusammenfassung)

- MP-lite statt echtem MP: **synthetische Punkte + Zufalls-Layouts**; die Kopplung an die echte Oviedo-MP-Web-Version steht noch aus.
- anndata/.h5ad-Schicht (numerische Messmatrizen) ist **noch nicht implementiert** — weder im Wrapper (to_anndata) noch im Mediator.
- Bulk-Download über gdc-client ist Platzhalter (status="not_run"); aktuell nur der Metadaten-Tier ist aktiv.
- Cache-Tiers 2 und 3 sind Grundgerüst; genutzt wird derzeit nur Tier 1 (Recipes).
- Alignment-Tabelle ist eine kleine, **verifizierte Testbasis** (3 Einträge), bewusst inkrementell.
- frontend/ ist leer; die Visualisierungstechnologie ist noch nicht final entschieden (kein Compose-Service).
- Compose-Service mediator existiert, wird von start_all aber nicht genutzt — dort läuft der Mediator lokal via uvicorn.
- Fuseki nutzt admin/admin — Prototyp-Credentials, später zu härten.
- Veralteter Docstring in enrichment.py („Alignment-Tabelle leer“) — sollte bei Gelegenheit aktualisiert werden.

10 Anhang

10.1 Ports & Dienste

Dienst	Port	Autor	Start
graph-db (Fuseki/TDB2)	3030	Marcel • Wissensnetz	docker compose up -d graph-db
Mediator (uvicorn)	8000	Kollege B • Mediator	uvicorn app.main:app
MP-lite (Bokeh)	5006	Marcel • Wissensnetz	bokeh serve mp_lite/app.py

10.2 Wichtige Umgebungsvariablen

- GRAPH_DB_URL / GRAPH_DB_HOST / GRAPH_DB_PORT — Fuseki-Adresse (im Compose-Netz <http://graph-db:3030>).
- GRAPH_DB_DATASET (Default databridge), GRAPH_DB_ADMIN_USER/PASSWORD (Default admin/admin).
- MEDIATOR_URL (Default <http://localhost:8000>), GDC_API_BASE_URL (Default <https://api.gdc.cancer.gov>).
- DATABRIDGE_ONTOLOGY_DIR, DATABRIDGE_CACHE_DIR — Pfad-Overrides für Container.

10.3 Namespaces

- db: = <http://databridge.hka/onto#> (TBox/Vokabular)
- Instanzen = [http://databridge.hka/instance/...](http://databridge.hka/instance/)
- ncit: = http://purl.obolibrary.org/obo/NCIT_ · prov: , oa: (Rückkanal)